

Phát hiện ransomware trên chuỗi khối Bitcoin bằng mạng nơ-ron hồi tiếp (LSTM/GRU) và mạng tích chập (CNN)

Đồ án cuối kỳ — Nhập môn Học máy

Nhóm: [Họ tên thành viên — MSSV]

Ngày 8 tháng 10 năm 2026

Tóm tắt nội dung

Đồ án áp dụng mạng nơ-ron hồi tiếp (LSTM, GRU) và mạng tích chập một chiều (CNN) cho bài toán phát hiện và phân loại ransomware trên bộ dữ liệu BitcoinHeist (2,9 triệu dòng, 7 lớp, lớp `white` chiếm 98,6%). Vì dữ liệu ở dạng bảng, chúng tôi so sánh hai cách biểu diễn chuỗi: (A) coi 11 đặc trưng của một dòng là một chuỗi 11 bước, và (B) dựng chuỗi lịch sử thật của mỗi địa chỉ qua các ngày (tối đa 16 lần xuất hiện), đánh giá trên cách chia theo địa chỉ để tránh rò rỉ. Hơn 40 cấu hình được thử nghiệm có hệ thống (số tầng, số units, hai chiều, cách gộp chuỗi, BatchNorm, Dropout, kích thước kernel, pooling, learning rate, batch size, độ dài lịch sử, các cấu hình chính chạy với 3 seed. Với biểu diễn A, RNN/CNN đạt macro F1 0,51–0,54 — ngang MLP và thấp hơn XGBoost (0,575): xếp đặc trưng thành chuỗi không tạo thêm thông tin. Với biểu diễn B, macro F1 tăng lên 0,72–0,74; mô hình nơ-ron tốt nhất là CNN kernel 5 (0,742), vượt XGBoost cùng thông tin một dòng (0,565) nhưng vẫn thấp hơn XGBoost có đặc trưng lịch sử tổng hợp (0,767). Phân tích chi tiết cho thấy RNN/CNN vượt XGBoost trên các địa chỉ có lịch sử dài (≥ 6 lần xuất hiện) nhưng thua trên các địa chỉ mới — nhóm chiếm 90% dữ liệu. Biểu diễn B dễ overfit hơn và được lợi rõ rệt từ BatchNorm + Dropout, trong khi với biểu diễn A điều chuẩn thêm chỉ gây underfit.

Mục lục

1	Giới thiệu bài toán và mục tiêu	3
2	Mô tả bộ dữ liệu	3
3	Tiền xử lý và chuẩn bị dữ liệu	4
4	Phương pháp và kiến trúc mô hình	5
4.1	Biểu diễn dữ liệu thành chuỗi	5
4.2	Mô hình RNN (LSTM / GRU)	6
4.3	Mô hình CNN (Conv1D)	6
4.4	Mô hình nền (baseline)	7
5	Thiết lập thực nghiệm	7
6	Kết quả thực nghiệm	8
6.1	Biểu diễn A — mỗi đặc trưng là một bước (chia ngẫu nhiên, so sánh với Bài 1)	8
6.2	Biểu diễn B — lịch sử của địa chỉ (chia theo địa chỉ)	11
6.3	Kết quả theo độ dài lịch sử	13
6.4	F1 từng lớp và ma trận nhầm lẫn	14
6.5	Tốc độ huấn luyện	14
6.6	Trả lời các câu hỏi của đề	15

7	Phân tích overfitting / underfitting	16
7.1	Biểu diễn A: giới hạn bởi thông tin, không phải bởi năng lực mô hình	17
7.2	Biểu diễn B: overfitting rõ hơn và vai trò của điều chuẩn	18
7.3	Biện pháp đã áp dụng và đề xuất	19
8	Hạn chế và hướng phát triển	19
8.1	Hạn chế	19
8.2	Hướng phát triển	20
	Tài liệu tham khảo	20
A	Mã nguồn và cách chạy lại	21

1 Giới thiệu bài toán và mục tiêu

Ransomware (mã độc tống tiền) mã hoá dữ liệu của nạn nhân và đòi tiền chuộc, thường được trả bằng Bitcoin. Vì mọi giao dịch Bitcoin đều công khai trên chuỗi khối, ta có thể quan sát hành vi của các địa chỉ nhận tiền và đặt câu hỏi: *chỉ từ các đặc trưng giao dịch của một địa chỉ trong một ngày, có thể nhận ra địa chỉ đó là địa chỉ bình thường hay thuộc một họ ransomware cụ thể không?*

Bài toán. Phân loại đa lớp (7 lớp): `white` (bình thường), 5 họ ransomware phổ biến nhất (`paduaCryptoWall`, `montrealCryptoLocker`, `princetonCerber`, `princetonLocky`, `montrealCryptXXX`) và `otherRansom` (gộp 23 họ hiếm còn lại). Cách định nghĩa nhãn, đặc trưng và cách chia dữ liệu được giữ nguyên như bài giữa kỳ (Bài 1) để các kết quả so sánh được với nhau.

Mục tiêu đề án.

- Xây dựng, huấn luyện và đánh giá mạng nơ-ron hồi tiếp (LSTM, GRU) và mạng tích chập 1 chiều (CNN Conv1D) cho bài toán trên.
- Vì dữ liệu ở dạng bảng, đề xuất và so sánh **hai cách biểu diễn chuỗi**: (A) coi 11 đặc trưng của một dòng là một chuỗi; (B) dựng *chuỗi lịch sử thật* của mỗi địa chỉ qua các ngày.
- Thực nghiệm có hệ thống (≈ 50 cấu hình, các cấu hình chính chạy 3 seed), so sánh với các mô hình nền (Logistic Regression, KNN, Random Forest, HistGradientBoosting, LightGBM, XGBoost, MLP), phân tích overfitting/underfitting và các lỗi dự đoán.
- Đánh giá trung thực: chỉ ra khi nào RNN/CNN *không* phải lựa chọn tốt nhất và vì sao.

2 Mô tả bộ dữ liệu

Nguồn. *BitcoinHeistRansomwareAddressDataset* (UCI Machine Learning Repository; Akcora và cộng sự [1]). Dữ liệu gồm **2.916.697 dòng**, 10 cột, giai đoạn 2011–2018, **2.631.095 địa chỉ** khác nhau. Mỗi dòng mô tả một địa chỉ Bitcoin trong một ngày (24 giờ), với các đặc trưng được tính từ đồ thị giao dịch (Bảng 1).

Bảng 1: Các cột của bộ dữ liệu.

Cột	Kiểu	Ý nghĩa
<code>address</code>	chuỗi	Địa chỉ Bitcoin (định danh, không dùng làm đặc trưng trực tiếp)
<code>year, day</code>	số nguyên	Năm (2011–2018) và ngày trong năm (1–365)
<code>length</code>	số	Độ dài chuỗi giao dịch dài nhất dẫn tới địa chỉ (0–144)
<code>weight</code>	số	Lượng “đầu vào” được gộp về địa chỉ
<code>count</code>	số	Số chuỗi giao dịch dẫn tới địa chỉ
<code>looped</code>	số	Số giao dịch tách ra rồi gộp lại (dấu hiệu “trộn” tiền)
<code>neighbors</code>	số	Số giao dịch có địa chỉ là đầu ra
<code>income</code>	số	Số satoshi nhận được ($1 \text{ BTC} = 10^8 \text{ satoshi}$), từ $3 \cdot 10^7$ đến $5 \cdot 10^{13}$
<code>label</code>	phân loại	<code>white</code> hoặc tên họ ransomware (28 họ)

Phân bố nhãn — mất cân bằng nghiêm trọng. Lớp `white` chiếm **98,58%**; 6 lớp ransomware cộng lại chỉ có 41.413 dòng (1,42%), lớp nhỏ nhất (`otherRansom`) chỉ 1.441 dòng (Bảng 2). Vì vậy *accuracy* gần như vô nghĩa (đoán toàn `white` đã đạt 98,6%); độ đo chính của đề án là **macro F1** (trung bình F1 của 7 lớp, mỗi lớp có trọng số như nhau), kèm F1 nhị phân “ransomware hay không”, *precision/recall* từng lớp và ma trận nhầm lẫn.

Bảng 2: Phân bố 7 lớp và đặc điểm “lặp lại” của địa chỉ (tính trên toàn bộ dữ liệu). *Lần đầu*: tỉ lệ dòng mà địa chỉ chưa từng xuất hiện trước đó; *trung vị số lần trước*: số lần địa chỉ đã xuất hiện ở các ngày trước.

Lớp	Số dòng	Tỉ lệ (%)	Số địa chỉ	Lần đầu (%)	Trung vị số lần trước
white	2.875.284	98,580	2.610.246	90,8	0
paduaCryptoWall	12.390	0,425	1.894	15,3	5
montrealCryptoLocker	9.315	0,319	1.509	16,2	13
princetonCerber	9.223	0,316	9.177	99,5	0
princetonLocky	6.625	0,227	6.584	99,4	0
montrealCryptXXX	2.419	0,083	1.354	56,0	0
otherRansom	1.441	0,049	331	23,0	6

Các đặc điểm quan trọng.

- **Đặc trưng lệch rất mạnh:** `income` trải dài 6 bậc độ lớn, `count/looped` có trung vị 1/0 nhưng tối đa $\approx 14.500 \Rightarrow$ cần biến đổi $\log(1+x)$ trước khi chuẩn hoá.
- **white** được lấy mẫu cố định 1.000 dòng/ngày (365.000 dòng mỗi năm), còn ransomware thì lấy đủ. Tỉ lệ ransomware thay đổi mạnh theo năm (0,001% năm 2018 đến 4,1% năm 2016) nên năm/tháng là thông tin hữu ích.
- **Địa chỉ có thể xuất hiện nhiều ngày** (Bảng 2). Đây là điểm mấu chốt cho biểu diễn chuỗi B: `CryptoLocker`, `CryptoWall`, `otherRansom` *tái sử dụng* địa chỉ rất nhiều (chỉ 15–23% dòng là lần đầu), trong khi `Cerber`, `Locky` gần như luôn dùng địa chỉ mới (99,5%), giống `white` (90,8%). Không có địa chỉ nào mang hai nhãn khác nhau.
- Hệ quả: nếu chia train/test ngẫu nhiên theo dòng, **cùng một địa chỉ ransomware có thể nằm ở cả train và test** (55% dòng ransomware trong test có địa chỉ đã xuất hiện trong train). Biểu diễn B vì vậy bắt buộc phải *chia theo địa chỉ*.

Dạng bài toán. Phân loại có giám sát, đa lớp, dữ liệu bảng, mất cân bằng nặng. Dữ liệu *không* phải ảnh, văn bản hay chuỗi thời gian “tự nhiên” — mục 4.1 trình bày cách chuyển đổi để dùng RNN và CNN.

3 Tiền xử lý và chuẩn bị dữ liệu

Làm sạch. Dữ liệu không có giá trị thiếu và không có dòng trùng lặp hoàn toàn; không cần điền khuyết. Cột `address` chỉ dùng để nhóm (dựng lịch sử, chia dữ liệu theo địa chỉ), không đưa trực tiếp vào mô hình.

Định nghĩa nhãn. 5 họ ransomware nhiều mẫu nhất giữ nguyên tên, 23 họ còn lại gộp thành `otherRansom` (nhiều họ chỉ có 1–60 dòng, không thể học riêng) $\Rightarrow$ 7 lớp.

Xây dựng đặc trưng (giống Bài 1).

- 6 đặc trưng gốc: `length`, `weight`, `count`, `looped`, `neighbors`, `income`.
- 5 đặc trưng mới: `income_tz` (số chữ số 0 ở cuối `income` — độ “tròn” của số tiền, tiền chuộc thường là số tròn), `income_per_count`, `income_per_neighbor`, `weight_per_count`, `looped_ratio`.
- Thời gian: `year`, `month`, `weekday` (tính từ `year + day`), mã hoá **one-hot** ($8 + 12 + 7 = 27$ chiều).

Chuẩn hoá. 11 đặc trưng số được biến đổi $x \mapsto \log(1+x)$ rồi chuẩn hoá $z = (x - \mu)/\sigma$, với μ, σ **chỉ tính trên tập train** (tránh rò rỉ thông tin từ validation/test).

Chia dữ liệu. Tỷ lệ train/validation/test = 64/16/20, cố định `random_state=42`, mọi mô hình trong cùng một nhóm thí nghiệm dùng *chung* một cách chia:

- **Chia ngẫu nhiên phân tầng theo nhãn** (dùng cho biểu diễn A, giống hệt Bài 1): train 1.866.685 / validation 466.672 / test 583.340 dòng (test có 8.283 dòng ransomware).
- **Chia theo địa chỉ** (`GroupShuffleSplit`; dùng cho biểu diễn B): mọi dòng của một địa chỉ nằm trọn trong một tập. Train 1.867.045 / validation 466.842 / test 582.810 dòng (7.911 dòng ransomware trong test). Đã kiểm tra: không có địa chỉ nào xuất hiện ở cả train và test.

Xử lý mất cân bằng (giống Bài 1).

1. **Undersampling tập train:** giữ ngẫu nhiên 100.000 dòng `white` + toàn bộ dòng ransomware $\Rightarrow$ khoảng 126.500 dòng (ransomware chiếm $\approx 21\%$). Bài 1 đã chứng minh mức 100k cho macro F1 tốt nhất.
2. **Hiệu chỉnh trọng số lớp sau huấn luyện:** mô hình học trên phân bố đã cân bằng nên dự đoán “ngiên” về ransomware. Trên tập *validation* (phân bố thật), ta tìm vector $w \in \mathbb{R}^7$ (dò lưới theo từng lớp, 3 vòng) sao cho $\hat{y} = \arg \max_k p_k w_k$ có macro F1 cao nhất; w sau đó được giữ cố định khi đánh giá test.

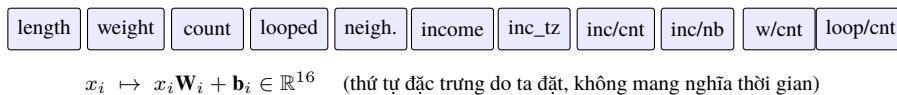
Tập validation còn được dùng ở dạng **val_us** — `undersample white` cùng tỷ lệ với tập train (31.626 dòng) — để đường cong loss/F1 của train và validation *so sánh được với nhau* và để *early stopping*. **Tập test chỉ được dùng một lần**, sau khi mô hình, epoch tốt nhất và trọng số w đã được chọn hoàn toàn trên train/validation.

4 Phương pháp và kiến trúc mô hình

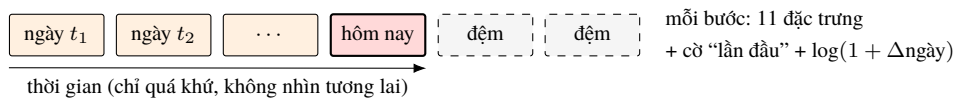
4.1 Biểu diễn dữ liệu thành chuỗi

Mỗi dòng của BitcoinHeist là một vector đặc trưng, không có cấu trúc chuỗi sẵn có. Đồ án thử hai cách chuyển đổi (Hình 1).

(A) Feature-as-sequence: 1 dòng $\rightarrow$ chuỗi 11 bước, bước i = đặc trưng thứ i



(B) Address history: dòng hiện tại + tối đa $L - 1$ lần xuất hiện *trước đó* của cùng địa chỉ

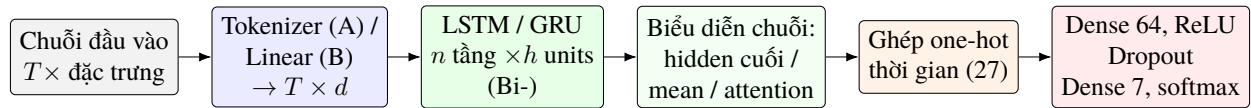


Hình 1: Hai cách biểu diễn một dòng dữ liệu bảng thành chuỗi để đưa vào RNN/CNN.

Biểu diễn A — “feature-as-sequence”. 11 đặc trưng số (đã chuẩn hoá) được xem là chuỗi độ dài cố định 11. Mỗi giá trị vô hướng x_i được “nhúng” thành vector $\mathbf{e}_i = x_i \mathbf{W}_i + \mathbf{b}_i \in \mathbb{R}^{16}$, trong đó $\mathbf{W}_i, \mathbf{b}_i$ là tham số riêng của từng đặc trưng (*feature tokenizer* [8]) — nhờ đó mô hình biết bước i là đặc trưng nào. Không có chuỗi dài ngắn khác nhau nên không cần đệm. 27 chiều one-hot thời gian không đưa vào chuỗi mà được *ghép vào trước lớp phân loại*. **Hạn chế:** thứ tự đặc trưng là tùy ý, không có “thời gian” hay “vị trí” thật; giả thiết cục bộ của CNN (đặc trưng kề nhau liên quan nhau) và giả thiết tuần tự của RNN chỉ đúng một phần (các đặc trưng tỉ lệ được đặt cạnh đặc trưng gốc tương ứng). Do đó ta *không kỳ vọng* RNN/CNN vượt mô hình cây trên biểu diễn này.

Biểu diễn B — “address history”. Với mỗi dòng (địa chỉ a , ngày t), lấy tối đa $L = 16$ lần xuất hiện gần nhất của a tại các ngày $\leq t$, sắp theo thời gian, bước cuối là chính dòng hiện tại. Mỗi bước là vector 13 chiều (11 đặc trưng + cờ “lần đầu xuất hiện” + $\log(1 + \text{số ngày từ lần trước})$), chiếu tuyến tính sang $\mathbb{R}^{32}$. **Chuỗi có độ dài khác nhau** (1–16): đệm 0 ở cuối, kèm mặt nạ (mask); RNN dùng `pack_padded_sequence` để bỏ qua phần đệm, CNN nhân mặt nạ trước mỗi lớp tích chập và dùng pooling có mặt nạ. Đây là chuỗi thời gian *thật*, chỉ dùng thông tin quá khứ (có thể triển khai thực tế: khi một địa chỉ nhận tiền hôm nay, ta đã biết lịch sử của nó). Phải chia dữ liệu theo địa chỉ để mô hình không “nhớ” địa chỉ đã thấy khi train.

4.2 Mô hình RNN (LSTM / GRU)

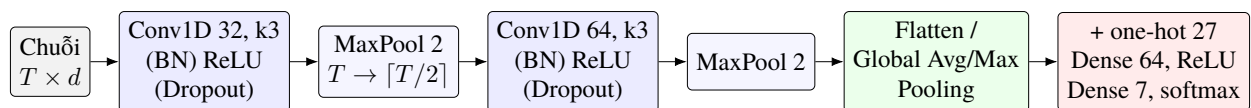


Hình 2: Kiến trúc tổng quát của mô hình RNN.

- **Lớp hồi tiếp:** LSTM (cổng quên/vào/ra + ô nhớ c_t) hoặc GRU (cổng cập nhật/đặt lại, không có ô nhớ riêng, ít tham số hơn $\approx 25\%$). Cấu hình gốc: 1 tầng, 64 hidden units, một chiều. Các biến thể: 2 tầng, 128 units, hai chiều (Bidirectional), không dropout.
- **Biểu diễn cuối của chuỗi:** hidden state cuối của tầng trên cùng (mặc định; với Bi-RNN ghép trạng thái cuối của hai chiều), trung bình (mean pooling) trên các bước thật, hoặc attention đơn giản $\alpha_t = \text{softmax}_t(\mathbf{v}^\top \mathbf{h}_t)$, $\mathbf{z} = \sum_t \alpha_t \mathbf{h}_t$. Ở biểu diễn B, nhờ `pack_padded_sequence` hidden state cuối chính là trạng thái tại ngày hiện tại.
- **Dropout:** 0,2 ở lớp Dense cuối và giữa các tầng RNN khi có 2 tầng. Không dùng recurrent dropout (dropout trên kết nối hồi tiếp) vì cài đặt cuDNN của PyTorch không hỗ trợ; với 1 tầng và chuỗi ngắn, dropout ở đầu ra là đủ.
- **Đầu ra:** Dense 7 + softmax; hàm mất mát cross-entropy.

Chọn cả LSTM và GRU để so sánh kiến trúc, tốc độ và kết quả (mục 6).

4.3 Mô hình CNN (Conv1D)



Hình 3: Kiến trúc tổng quát của mô hình CNN 1 chiều.

- **Loại:** Conv1D (trượt dọc trực chuỗi, mỗi bước có d kênh = chiều embedding). Conv2D không phù hợp vì dữ liệu không có cấu trúc 2 chiều.
- **Lớp tích chập:** 2 lớp (32 và 64 filters) mặc định; biến thể 3 lớp (32-64-128), rộng (64-128), kernel 3 hoặc 5, padding “same”. Mỗi filter là một bộ phát hiện mẫu cục bộ trên k bước liên tiếp, dùng chung trọng số ở mọi vị trí (chia sẻ tham số) — với biểu diễn A là tổ hợp của k đặc trưng kề nhau, với B là mẫu hành vi trong k lần xuất hiện liên tiếp.
- **Pooling:** MaxPooling (biến thể AveragePooling) kích thước 2 sau mỗi lớp khi chuỗi còn dài ≥ 4 : giảm một nửa độ dài $\Rightarrow$ giảm số tham số của lớp Dense phía sau và tính toán, đồng thời giúp đặc trưng ít nhạy với dịch chuyển nhỏ (bất biến cục bộ).
- **Batch Normalization và Dropout** (0,3): bật/tắt để so sánh CNN đơn giản với CNN có điều chuẩn.

- **Chuyển sang lớp phân loại:** *Flatten* (giữ thông tin vị trí — hợp với A vì mỗi vị trí là một đặc trưng cố định) hoặc *Global Average/Max Pooling* (bất biến độ dài — bắt buộc với B vì chuỗi dài ngắn khác nhau).
- **Phần fully connected:** ghép one-hot thời gian $\rightarrow$ Dense 64 ReLU $\rightarrow$ Dropout $\rightarrow$ Dense 7 softmax.

4.4 Mô hình nền (baseline)

- Từ Bài 1 (cùng cách chia ngẫu nhiên, cùng tiền xử lý): Logistic Regression, KNN, Random Forest, HistGradientBoosting, LightGBM, XGBoost và Ensemble.
- **MLP** (2 tầng ẩn 128 units, ReLU, Dropout 0,2): cùng đặc trưng nhưng *không* coi là chuỗi — dùng để tách bạch lợi ích của “cấu trúc chuỗi” khỏi lợi ích của “mạng nơ-ron”.
- **XGBoost trên cách chia theo địa chỉ** với 3 mức thông tin lịch sử: chỉ dòng hiện tại (`XGB_current`); thêm số lần đã xuất hiện + khoảng cách ngày (`XGB_count`); thêm trung bình/max của 11 đặc trưng trên 15 lần xuất hiện trước (`XGB_hist`). Đây là đối thủ “dạng bảng” công bằng của biểu diễn B.

5 Thiết lập thực nghiệm

Môi trường. PyTorch 2.11 (CUDA 12.8), GPU NVIDIA RTX PRO 4000 Blackwell 24 GB (máy chủ Trainbox, Windows), Python 3.12. Toàn bộ dữ liệu của tập train/validation/test được đặt sẵn trên GPU; batch được cắt theo chỉ số dòng. Mã nguồn: thư mục `final/` (`data.py`, `models.py`, `configs.py`, `train.py`, `baselines.py`, `report.py`) và notebook `final_project.ipynb`.

Huấn luyện (chung cho mọi mạng nơ-ron).

- Hàm mất mát: **cross-entropy** trên đầu ra softmax 7 lớp. Thuật toán tối ưu: **Adam**, learning rate 10^{-3} (biến thể $3 \cdot 10^{-4}$), batch size 512 (biến thể 2048), cắt gradient theo chuẩn ≤ 1 .
- Tối đa 100 epoch, **early stopping** theo macro F1 trên `val_us` với patience 10; giữ lại trọng số của epoch tốt nhất.
- Sau huấn luyện: tìm trọng số lớp w trên toàn bộ validation $\rightarrow$ đánh giá test bằng $\arg \max(p \cdot w)$ (mục 3).
- Seed 42 cho mọi cấu hình; các cấu hình chính được chạy lại với seed 43, 44 để ước lượng độ dao động (báo cáo trung bình $\pm$ độ lệch chuẩn). Thay đổi seed chỉ thay đổi khởi tạo trọng số và thứ tự batch — tập dữ liệu giữ nguyên.

Thiết kế thực nghiệm. Mỗi lần chỉ thay đổi *một* yếu tố so với cấu hình gốc của nhóm để thấy rõ ảnh hưởng:

- **RNN** (gốc: LSTM 1 tầng $\times$ 64 units, hidden state cuối, dropout 0,2): LSTM $\leftrightarrow$ GRU; 1 $\leftrightarrow$ 2 tầng; 64 $\leftrightarrow$ 128 units; một chiều $\leftrightarrow$ hai chiều; hidden cuối $\leftrightarrow$ mean $\leftrightarrow$ attention; dropout 0,2 $\leftrightarrow$ 0; learning rate; batch size; độ dài lịch sử $L \in \{1, 4, 16, 32\}$ (biểu diễn B).
- **CNN** (gốc A: Conv 32-64, kernel 3, MaxPool, Flatten, không BN/Dropout): + BN; + Dropout; + BN + Dropout; Flatten $\leftrightarrow$ GAP; 2 $\leftrightarrow$ 3 lớp; số filters; kernel 3 $\leftrightarrow$ 5; Max $\leftrightarrow$ Average pooling; learning rate; batch size. Biểu diễn B dùng GAP (hoặc Global Max) vì chuỗi có độ dài thay đổi.
- **Đối chứng:** MLP (không coi là chuỗi); `B_gru_L1` (biểu diễn B nhưng chỉ 1 bước = không có lịch sử); các biến thể `_cnt` thêm đặc trưng đếm vào đầu vào phụ để so sánh công bằng với `XGB_count`.

Độ đo. Macro F1 (chính), F1 nhị phân ransomware/white, accuracy, precision/recall/F1 có trọng số và macro, F1 từng lớp, ma trận nhầm lẫn; thời gian huấn luyện (tổng và trung bình mỗi epoch) và số tham số.

6 Kết quả thực nghiệm

Tất cả số liệu là trên **tập test**, sau khi chọn epoch (early stopping trên `val_us`) và trọng số lớp w (trên validation). Với cấu hình chạy 3 seed (42, 43, 44), bảng ghi trung bình $\pm$ độ lệch chuẩn của macro F1; các cột khác lấy trung bình. Cột “Ep.” là *epoch tốt nhất / số epoch đã chạy* (early stopping dừng sau 10 epoch không cải thiện).

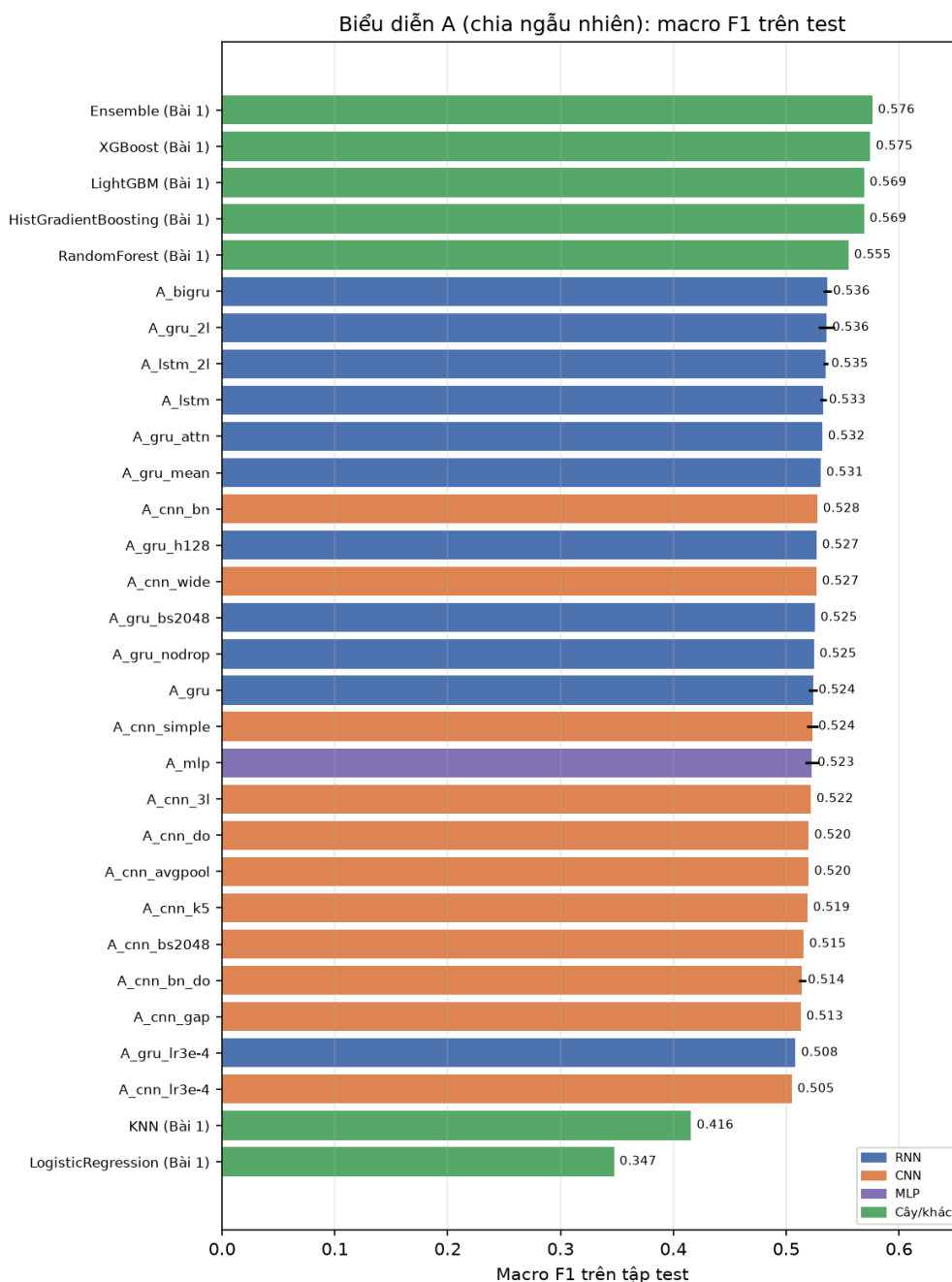
6.1 Biểu diễn A — mỗi đặc trưng là một bước (chia ngẫu nhiên, so sánh với Bài 1)

Bảng 3: Biểu diễn A — các cấu hình RNN (chia ngẫu nhiên). Ep.: epoch tốt nhất/số epoch đã chạy; s/ep.: giây mỗi epoch khi chạy song song (xem Bảng 9 cho tốc độ đo riêng).

Cấu hình	Kiến trúc chính	LR	Batch	Ep.	Tham số	s/ep.	Macro F1	F1 nhị phân	Nhận xét
A_bigr	BiGRU 1x64, last	1e-3	512	30/40	42.279	1.55	0.5365 $\pm$ 0.0037	0.4675	hội tụ sớm hơn, cao nhất nhóm A (trong dao động seed)
A_gru_2l	GRU 2x64, last	1e-3	512	49/59	47.399	1.49	0.5358 $\pm$ 0.0073	0.4614	thêm tầng: cải thiện rất nhỏ
A_lstm_2l	LSTM 2x64, last	1e-3	512	49/59	60.967	0.81	0.5350 $\pm$ 0.0024	0.4631	thêm tầng: chênh lệch trong mức dao động seed
A_lstm	LSTM 1x64, last	1e-3	512	50/60	27.687	0.71	0.5329 $\pm$ 0.0032	0.4619	cấu hình gốc RNN
A_gru_attn	GRU 1x64, attn	1e-3	512	47/57	22.504	1.64	0.5320	0.4743	attention $\approx$ hidden cuối
A_gru_mean	GRU 1x64, mean	1e-3	512	42/52	22.439	1.49	0.5305	0.4607	mean pooling $\approx$ hidden cuối
A_gru_h128	GRU 1x128, last	1e-3	512	35/45	66.855	1.53	0.5268	0.4645	gấp 3 tham số, không tốt hơn
A_gru_bs2048	GRU 1x64, last	1e-3	2048	98/100	22.439	0.40	0.5254	0.4625	batch lớn: ít bước cập nhật, chạm giới hạn 100 epoch
A_gru_nodrop	GRU 1x64, last, do 0.0	1e-3	512	63/73	22.439	1.45	0.5245	0.4550	bỏ dropout: khoảng cách train-val lớn hơn
A_gru	GRU 1x64, last	1e-3	512	42/52	22.439	0.76	0.5237 $\pm$ 0.0040	0.4606	ít tham số hơn LSTM 19%, thấp hơn nhẹ
A_gru_lr3e-4	GRU 1x64, last	3e-4	512	84/94	22.439	1.49	0.5082	0.4422	lr nhỏ: hội tụ chậm, underfit

Bảng 4: Biểu diễn A — các cấu hình CNN và MLP (chia ngẫu nhiên).

Cấu hình	Kiến trúc chính	LR	Batch	Ep.	Tham số	s/ep.	Macro F1	F1 nhị phân	Nhận xét
A_cnn_bn	Conv 32-64 k3 maxpool +BN, flatten, do 0.0	1e-3	512	53/63	22.855	1.02	0.5276	0.4448	BN: hội tụ nhanh nhất, overfit nhẹ
A_cnn_wide	Conv 64-128 k3 maxpool +BN, flatten, do 0.3	1e-3	512	42/52	55.399	1.30	0.5267	0.4538	gấp đôi filters, không tốt hơn
A_cnn_simple	Conv 32-64 k3 maxpool, flatten, do 0.0	1e-3	512	47/57	22.663	0.95	0.5235 ± 0.0050	0.4513	cấu hình gốc CNN
A_mlp	MLP 2x128, do 0.2	1e-3	512	42/52	22.407	0.97	0.5227 ± 0.0063	0.4521	không coi là chuỗi — ngang RNN/CNN
A_cnn_3l	Conv 32-64-128 k3 maxpool +BN, flatten, do 0.3	1e-3	512	42/52	60.103	1.34	0.5219	0.4554	sâu hơn không giúp (chuỗi chỉ 11 bước)
A_cnn_do	Conv 32-64 k3 maxpool, flatten, do 0.3	1e-3	512	42/52	22.663	0.96	0.5199	0.4544	dropout: train $\approx$ val, hơi underfit
A_cnn_avgpool	Conv 32-64 k3 avgpool +BN, flatten, do 0.3	1e-3	512	42/52	22.855	1.56	0.5199	0.4512	AvgPool $\approx$ MaxPool
A_cnn_k5	Conv 32-64 k5 maxpool +BN, flatten, do 0.3	1e-3	512	42/52	27.975	1.15	0.5188	0.4496	kernel 5 $\approx$ kernel 3
A_cnn_bs2048	Conv 32-64 k3 maxpool +BN, flatten, do 0.3	1e-3	2048	73/83	22.855	0.41	0.5154	0.4387	batch lớn: hội tụ chậm hơn
A_cnn_bn_do	Conv 32-64 k3 maxpool +BN, flatten, do 0.3	1e-3	512	42/52	22.855	1.11	0.5141 ± 0.0033	0.4424	điều chuẩn thừa với mô hình nhỏ
A_cnn_gap	Conv 32-64 k3 maxpool +BN, gap, do 0.3	1e-3	512	45/55	14.663	1.12	0.5129	0.4427	GAP bỏ thông tin “vị trí = đặc trưng nào”
A_cnn_lr3e-4	Conv 32-64 k3 maxpool +BN, flatten, do 0.3	3e-4	512	53/63	22.855	1.39	0.5049	0.4390	lr nhỏ: underfit



Hình 4: Biểu diễn A: macro F1 trên test của mọi cấu hình RNN/CNN/MLP và các mô hình của Bài 1 (cùng cách chia, cùng tiền xử lý). Vạch đen: độ lệch chuẩn qua 3 seed.

Nhận xét.

- Mọi cấu hình RNN nằm trong khoảng macro F1 0,51–0,54, CNN trong khoảng 0,50–0,53. Tốt nhất là A_bigru, A_gru_2l, A_lstm_2l ($\approx 0,535$ –0,537) và A_cnn_bn (0,528). Độ lệch chuẩn giữa các seed là 0,002–0,007, nên phần lớn khác biệt giữa các biến thể RNN (LSTM/GRU, 1/2 tầng, một/hai chiều, last/mean/attention) **không có ý nghĩa thống kê**.
- MLP** (không coi dữ liệu là chuỗi) đạt $\approx 0,52$ — ngang RNN/CNN. Việc “xếp 11 đặc trưng thành chuỗi” không mang lại thông tin mới: thứ tự các đặc trưng là tùy ý, phép tích chập trên “đặc trưng kề nhau” và trạng thái ẩn “truyền từ đặc trưng này sang đặc trưng kia” không có ý nghĩa vật lý. Đúng như đề bài cảnh báo ở mục 14.
- XGBoost của Bài 1 (0,575) vẫn tốt hơn mọi mạng nơ-ron trên biểu diễn A** (kể cả Random Forest

0,555 cũng hơn). Điều này khớp với kết quả đã biết: với dữ liệu bảng có đặc trưng lệch phân phối mạnh và quan hệ dạng ngưỡng, mô hình cây vẫn mạnh hơn mạng nơ-ron [9]. Mạng nơ-ron chỉ vượt Logistic Regression (0,347) và KNN (0,416).

- Learning rate nhỏ ($3 \cdot 10^{-4}$) cho kết quả kém nhất ở cả RNN và CNN (underfit, xem mục 7); batch 2048 làm số bước cập nhật mỗi epoch giảm 4 lần, mô hình GRU chạm giới hạn 100 epoch trước khi hội tụ.
- Với CNN, thêm BatchNorm giúp hội tụ nhanh và cho kết quả tốt nhất; Dropout (0,3) và GAP làm giảm kết quả — mô hình ≈ 23 nghìn tham số trên 126 nghìn mẫu không thừa năng lực, điều chuẩn thêm chỉ làm nó underfit. Flatten tốt hơn GAP vì ở biểu diễn A vị trí chính là *danh tính của đặc trưng*.

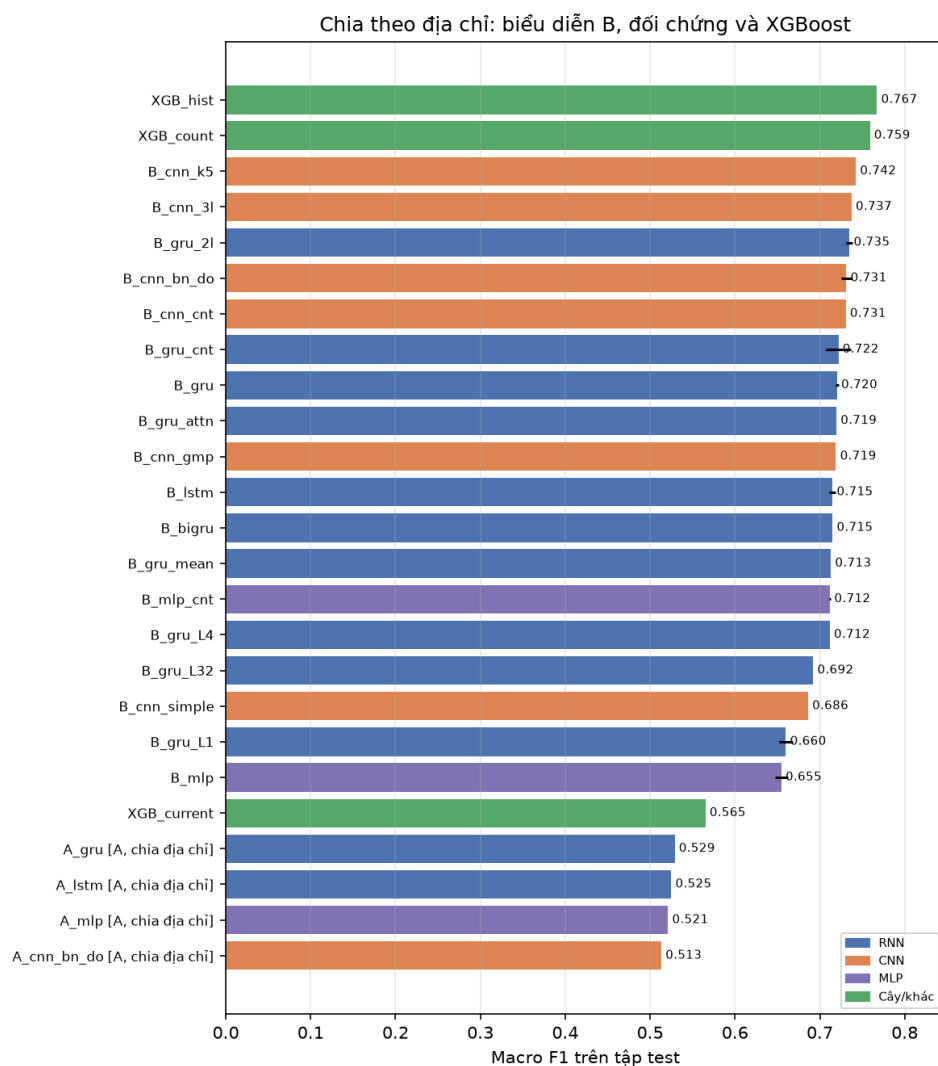
6.2 Biểu diễn B — lịch sử của địa chỉ (chia theo địa chỉ)

Bảng 5: Biểu diễn B — lịch sử địa chỉ (chia theo địa chỉ), $L = 16$ nếu không ghi.

Cấu hình	Kiến trúc chính	LR	Batch	Ep.	Tham số	s/ep.	Macro F1	F1 nhị phân	Nhận xét
B_cnn_k5	Conv 32-64 k5 maxpool +BN, gap, do 0.3	1e-3	512	66/76	22.439	1.23	0.7424	0.7068	kernel 5 bao phủ nhiều lần xuất hiện hơn
B_cnn_3l	Conv 32-64-128 k3 maxpool +BN, gap, do 0.3	1e-3	512	32/42	45.351	1.54	0.7373	0.6973	sâu hơn có ích với chuỗi 16 bước
B_gru_2l	GRU 2x64, last	1e-3	512	53/63	50.567	4.68	0.7349 ± 0.0036	0.7140	RNN tốt nhất trên B, dropout giữa tầng
B_cnn_bn_do	Conv 32-64 k3 maxpool +BN, gap, do 0.3	1e-3	512	22/32	16.295	1.20	0.7313 ± 0.0064	0.6932	cấu hình gốc CNN-B
B_cnn_cnt	Conv 32-64 k3 maxpool +BN, gap, do 0.3, +đếm	1e-3	512	20/30	16.487	1.10	0.7305	0.6994	+ đặc trưng đếm: không cải thiện
B_gru_cnt	GRU 1x64, last, +đếm	1e-3	512	69/79	25.799	2.41	0.7219 ± 0.0145	0.7011	+ đếm không giúp GRU (đã tự đếm được)
B_gru	GRU 1x64, last	1e-3	512	53/63	25.607	4.03	0.7202 ± 0.0022	0.6980	cấu hình gốc RNN-B
B_gru_attn	GRU 1x64, attn	1e-3	512	26/36	25.672	6.23	0.7194	0.6921	attention $\approx$ hidden cuối
B_cnn_gmp	Conv 32-64 k3 maxpool +BN, gmp, do 0.3	1e-3	512	22/32	16.295	1.41	0.7189	0.6755	Global Max $\approx$ GAP
B_lstm	LSTM 1x64, last	1e-3	512	60/70	31.879	1.75	0.7150 ± 0.0038	0.6998	$\approx$ GRU
B_bigr	BiGRU 1x64, last	1e-3	512	31/41	48.519	10.46	0.7149	0.6907	hai chiều không giúp (bước cuối quan trọng nhất)
B_gru_mean	GRU 1x64, mean	1e-3	512	42/52	25.607	5.81	0.7130	0.6920	mean làm loãng bước hiện tại
B_mlp_cnt	MLP 2x128, do 0.2, +đếm	1e-3	512	44/54	23.047	0.71	0.7121 ± 0.0011	0.6880	MLP + đếm $\approx$ GRU có lịch sử
B_gru_L4	GRU 1x64, last, L=4	1e-3	512	59/69	25.607	2.31	0.7116	0.6909	4 bước đã cho phần lớn lợi ích
B_gru_L32	GRU 1x64, last, L=32	1e-3	512	26/36	25.607	5.66	0.6921	0.6886	dừng sớm (epoch 26), không tốt hơn L=16
B_cnn_simple	Conv 32-64 k3 maxpool, gap, do 0.0	1e-3	512	12/22	16.103	1.13	0.6861	0.6518	không BN/Dropout: overfit rất sớm
B_gru_L1	GRU 1x64, last, L=1	1e-3	512	41/51	25.607	2.79	0.6600 ± 0.0079	0.6374	1 bước: không có chuỗi
B_mlp	MLP 2x128, do 0.2	1e-3	512	88/98	22.663	0.88	0.6546 ± 0.0074	0.6368	chỉ bước hiện tại (có cờ lần đầu, Δ ngày)

Bảng 6: Đối chứng trên cùng cách chia theo địa chỉ: XGBoost với các mức thông tin lịch sử và biểu diễn A.

Cấu hình	Kiến trúc chính	LR	Batch	Ep.	Tham số	s/ep.	Macro F1	F1 nhị phân	Nhận xét
A_gru	GRU 1x64, last	1e-3	512	48/58	22.439	1.28	0.5289	0.4496	A trên chia theo địa chỉ
A_lstm	LSTM 1x64, last	1e-3	512	40/50	27.687	2.11	0.5249	0.4496	A trên chia theo địa chỉ
A_mlp	MLP 2x128, do 0.2	1e-3	512	65/75	22.407	0.84	0.5210	0.4456	A trên chia theo địa chỉ
A_cnn_bn_do	Conv 32-64 k3 maxpool +BN, flatten, do 0.3	1e-3	512	60/70	22.855	1.06	0.5132	0.4472	A trên chia theo địa chỉ
XGB_hist	XGBoost + đếm + mean/max lịch sử	–	–	–	–	–	0.7673	0.7502	tốt nhất toàn bộ thực nghiệm
XGB_count	XGBoost + đếm lịch sử	–	–	–	–	–	0.7589	0.7437	+ 3 đặc trưng đếm
XGB_current	XGBoost, đồng hiện tại	–	–	–	–	–	0.5651	0.4966	không có thông tin lịch sử



Hình 5: Cùng cách chia theo địa chỉ: biểu diễn B, các biến thể đối chứng, XGBoost với 3 mức thông tin lịch sử và biểu diễn A.

Nhận xét.

- **Biểu diễn B tăng macro F1 từ $\approx 0,53$ lên $\approx 0,72-0,74$** so với chính các mô hình đó trên biểu diễn A (cùng cách chia theo địa chỉ, Bảng 6). Đây là thông tin thật sự mới mà một dòng đơn lẻ không có: một địa chỉ ransomware (CryptoLocker, CryptoWall) nhận tiền nhiều ngày liên tiếp với mẫu hành vi lặp lại.
- **Mô hình nơ-ron tốt nhất là CNN:** B_cnn_k5 (0,742) và B_cnn_3l (0,737); RNN tốt nhất là B_gru_2l (0,735 $\pm$ 0,004). Trên B, *tăng trường nhìn* (kernel 5, thêm tầng) có ích vì chuỗi dài hơn

(tối 16 bước) và mẫu hành vi trải dài nhiều lần xuất hiện.

- **Phần lợi ích do chuỗi mang lại:** B_gru_L1 (chỉ bước hiện tại, nhưng có cờ “lần đầu” và khoảng cách ngày) đạt 0,660; L=4 đạt 0,712 và L=16 đạt 0,720. So với biểu diễn A trên cùng cách chia ($\approx 0,53$), riêng việc *biết địa chỉ từng xuất hiện* đã cho +0,13, và việc *nhìn thấy nội dung các lần xuất hiện trước* cho thêm +0,06–0,08. L=32 không tốt hơn (0,692; dừng sớm ở epoch 26): địa chỉ có >16 lần xuất hiện rất ít, phần đệm dài chỉ thêm nhiều.
- **Đôi chứng công bằng với XGBoost:** XGBoost chỉ dùng dòng hiện tại đạt 0,565; thêm 3 đặc trưng đếm (số lần đã xuất hiện, cờ lần đầu, khoảng cách ngày) đạt 0,759; thêm trung bình/max lịch sử đạt **0,767 — tốt nhất toàn bộ đồ án**. Khi được cho cùng thông tin lịch sử dưới dạng đặc trưng tổng hợp thủ công, mô hình cây vẫn hơn RNN/CNN $\approx 0,025$.
- Biến thể _cnt (thêm chính 3 đặc trưng đếm vào đầu vào phụ) không giúp GRU (0,722 so với 0,720, trong độ dao động seed) và không giúp CNN (0,731 so với 0,731): RNN/CNN *đã tự học được* thông tin đếm từ chuỗi. Ngược lại, MLP cần nó (0,655 $\rightarrow$ 0,712).
- LSTM và GRU cho kết quả gần nhau, chênh lệch $\leq 0,01$ và không nhất quán về chiều (A 1 tầng: LSTM hơn 0,009; A 2 tầng: bằng nhau; B: GRU hơn 0,005) — không có loại nào vượt trội rõ ràng. Bi-GRU không giúp ở B: bước cuối (ngày hiện tại) là quan trọng nhất và đã được RNN một chiều đọc sau cùng. Mean pooling kém hơn hidden cuối vì làm loãng bước hiện tại.

6.3 Kết quả theo độ dài lịch sử

Bảng 7 tách tập test theo số lần địa chỉ đã xuất hiện (tính cả ngày hiện tại).

Bảng 7: F1 nhị phân (ransomware/white) trên tập test theo số lần địa chỉ đã xuất hiện tính đến ngày dự đoán (chia theo địa chỉ). Hai hàng đầu: số dòng và tỉ lệ ransomware trong từng nhóm.

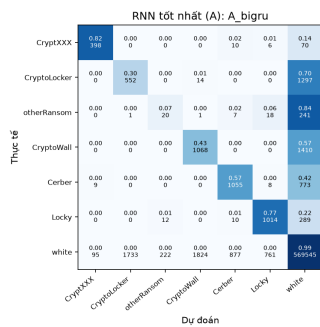
Số lần xuất hiện	1	2	3–5	6–15	>15
Số dòng test	526.219	19.404	17.092	13.076	7.019
% ransomware	0.79	2.64	4.58	7.55	20.69
XGB_current	0.504	0.469	0.540	0.524	0.414
B_mlp	0.559	0.428	0.694	0.833	0.844
B_gru_L1	0.545	0.420	0.678	0.831	0.867
B_gru	0.549	0.610	0.796	0.917	0.973
B_cnn_k5	0.547	0.625	0.812	0.943	0.948
B_mlp_cnt	0.553	0.604	0.738	0.854	0.929
XGB_hist	0.648	0.692	0.793	0.885	0.937

- Với địa chỉ **đã xuất hiện ≥ 3 lần**, RNN/CNN có lịch sử vượt XGBoost–lịch sử: ví dụ nhóm 6–15 lần F1 0,943 (B_cnn_k5) so với 0,885; nhóm >15 lần 0,973 (B_gru) so với 0,937. Mô hình chuỗi khai thác được *chi tiết từng lần xuất hiện* mà mean/max thủ công làm mất.
- Với địa chỉ **xuất hiện lần đầu** (90% số dòng test, 53% số dòng ransomware), không có lịch sử nào để học, và XGBoost tốt hơn rõ (0,648 so với $\approx 0,55$): mô hình cây khai thác đặc trưng của dòng hiện tại tốt hơn mạng nơ-ron (đúng như kết quả ở biểu diễn A). Vì nhóm này chiếm đa số, XGBoost thắng trên toàn bộ tập test.
- Đường B_gru_L1/B_mlp (không thấy nội dung lịch sử) tăng theo số lần xuất hiện chỉ nhờ cờ “lần đầu” và Δ ngày; B_mlp_cnt (biết số lần đã xuất hiện) cao hơn, và RNN/CNN thấy toàn bộ chuỗi cao nhất ở các nhóm dài.

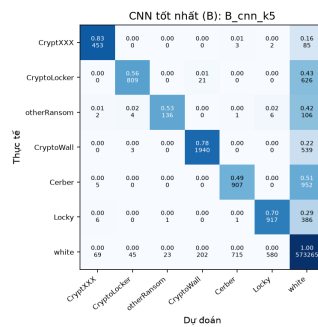
6.4 F1 từng lớp và ma trận nhầm lẫn

Bảng 8: F1 từng lớp trên tập test của các mô hình tiêu biểu (seed 42).

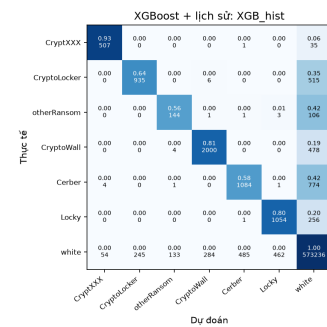
Mô hình	CryptXXX	CryptoLocker	otherRansom	CryptoWall	Cerber	Locky	white	Macro
XGBoost (A, ngẫu nhiên)	0.859	0.284	0.109	0.415	0.631	0.732	0.992	0.575
RNN tốt nhất (A)	0.807	0.266	0.074	0.397	0.555	0.648	0.992	0.534
CNN tốt nhất (A)	0.806	0.237	0.084	0.381	0.554	0.640	0.991	0.528
MLP (A)	0.780	0.249	0.077	0.385	0.522	0.629	0.991	0.519
XGBoost, không lịch sử	0.887	0.240	0.084	0.381	0.634	0.736	0.993	0.565
GRU, L=1 (chỉ cờ lần đầu + Δ ngày)	0.790	0.496	0.364	0.753	0.518	0.643	0.995	0.651
RNN tốt nhất (B)	0.858	0.678	0.578	0.806	0.556	0.673	0.996	0.735
CNN tốt nhất (B)	0.840	0.698	0.655	0.835	0.520	0.651	0.996	0.742
XGBoost + lịch sử	0.915	0.709	0.536	0.838	0.631	0.745	0.997	0.767



(a) RNN tốt nhất (A)



(b) CNN tốt nhất (B)



(c) XGBoost + lịch sử

Hình 6: Ma trận nhầm lẫn trên tập test (chuẩn hoá theo hàng = recall từng lớp). Các ma trận khác nằm trong thư mục `outputs/final/figures/` và notebook.

- Lịch sử địa chỉ cải thiện mạnh các họ **tái sử dụng địa chỉ**: CryptoLocker 0,26 $\rightarrow$ 0,70, CryptoWall 0,38 $\rightarrow$ 0,84, otherRansom 0,08 $\rightarrow$ 0,66 (CNN-B; ở lớp này CNN-B còn vượt XGBoost-lịch sử 0,536).
- Các họ **dùng địa chỉ mới mỗi lần** (Cerber, Locky: 99% dòng là lần đầu xuất hiện) không được lợi gì từ lịch sử; ở đây XGBoost hơn mạng nơ-ron (Cerber 0,63 so với 0,52–0,56; Locky 0,745 so với 0,65–0,67).
- Ma trận nhầm lẫn cho thấy lỗi **gần như hoàn toàn là nhầm giữa ransomware và white**; nhầm giữa các họ ransomware với nhau rất hiếm (55 trên 7.911 dòng ransomware với B_cnn_k5). Khó khăn của bài toán là *phát hiện*, không phải *phân biệt họ*.

Phân tích dự đoán sai (B_cnn_k5, chia theo địa chỉ). Mô hình bỏ sót 2.694/7.911 dòng ransomware (34%), trong đó 78% là địa chỉ xuất hiện lần đầu; 82% trong 1.634 báo động nhầm cũng rơi vào địa chỉ lần đầu. Tách theo họ: với CryptoLocker, CryptoWall và otherRansom, recall trên các dòng *lặp lại* là 0,69/0,92/0,72 nhưng trên các dòng *lần đầu* gần bằng 0 — mạng nơ-ron gần như chỉ nhận ra các họ này qua lịch sử, trong khi XGBoost-lịch sử vẫn bắt được 26%/11%/11% dòng lần đầu nhờ đặc trưng của chính giao dịch. Notebook (mục 10) liệt kê các dòng sai với độ tin cậy cao nhất và ví dụ chuỗi lịch sử tương ứng.

6.5 Tốc độ huấn luyện

Thời gian/epoch trong các bảng trên đo khi 6–8 tiến trình huấn luyện chạy song song trên cùng GPU, nên chỉ mang tính tham khảo. Bảng 9 đo lại *riêng từng mô hình* trên GPU rảnh.

Bảng 9: Tốc độ huấn luyện đo riêng (GPU không chia sẻ, trung bình 5 epoch sau 1 epoch khởi động, ≈ 126.500 dòng/epoch, batch 512).

Cấu hình	Kiến trúc	Tham số	Giây/epoch
A_lstm	LSTM 1x64, last	27.687	0.41
A_gru	GRU 1x64, last	22.439	0.39
A_lstm_2l	LSTM 2x64, last	60.967	0.65
A_gru_2l	GRU 2x64, last	47.399	0.53
A_bigru	BiGRU 1x64, last	42.279	0.48
A_cnn_simple	Conv 32-64 k3 maxpool, flatten, do 0.0	22.663	0.43
A_cnn_bn_do	Conv 32-64 k3 maxpool +BN, flatten, do 0.3	22.855	0.55
A_cnn_3l	Conv 32-64-128 k3 maxpool +BN, flatten, do 0.3	60.103	0.73
A_mlp	MLP 2x128, do 0.2	22.407	0.29
B_lstm	LSTM 1x64, last	31.879	0.92
B_gru	GRU 1x64, last	25.607	0.89
B_bigru	BiGRU 1x64, last	48.519	2.14
B_cnn_bn_do	Conv 32-64 k3 maxpool +BN, gap, do 0.3	16.295	0.68
B_cnn_3l	Conv 32-64-128 k3 maxpool +BN, gap, do 0.3	45.351	0.77
B_mlp	MLP 2x128, do 0.2	22.663	0.33

- **MLP nhanh nhất** (0,29–0,33 s/epoch): không có chiều chuỗi.
- Ở biểu diễn A (chuỗi chỉ 11 bước, độ dài cố định), RNN 1 tầng và CNN 2 lớp có tốc độ *gần như nhau* (0,39–0,43 s/epoch): cuDNN tối ưu RNN rất tốt cho chuỗi ngắn, còn chi phí BatchNorm/Dropout làm CNN chậm thêm (0,55 s). Thêm tầng làm chậm cả hai (LSTM 2 tầng 0,65 s, CNN 3 lớp 0,73 s).
- Ở biểu diễn B (16 bước, độ dài thay đổi), **CNN nhanh hơn RNN $\approx 25\%$** (0,68 so với 0,89–0,92 s/epoch): tích chập tính song song trên mọi vị trí, còn RNN phải tính tuần tự và xử lý `pack_padded_sequence`. Bi-GRU chậm nhất (2,14 s/epoch, gấp 2,4 lần GRU một chiều) mà không tốt hơn.
- **GRU ít tham số hơn LSTM** (3 cổng so với 4: 22.439 so với 27.687 tham số ở cấu hình gốc A) và nhanh hơn một chút (0,39 so với 0,41 s; 0,53 so với 0,65 s khi 2 tầng).
- Như vậy ở biểu diễn B, CNN vừa **nhanh hơn** vừa **tốt hơn** RNN.

6.6 Trả lời các câu hỏi của đề

- **Mô hình tốt nhất?** Trong các mạng nơ-ron: B_cnn_k5 (CNN 1 chiều trên lịch sử địa chỉ, macro F1 0,742). Trên toàn bộ đồ án: XGBoost với đặc trưng lịch sử tổng hợp (0,767). Với cùng thông tin một dòng (biểu diễn A), XGBoost Bài 1 (0,575) tốt hơn mọi RNN/CNN ($\leq 0,54$).
- **Vì sao?** Yếu tố quyết định là *thông tin đầu vào*, không phải loại mạng: thêm lịch sử địa chỉ tăng $\approx 0,2$ macro F1, còn mọi thay đổi kiến trúc trong cùng một biểu diễn chỉ thay đổi $\leq 0,03$. CNN tốt hơn RNN một chút trên B vì mẫu “nhận tiền lặp lại vài ngày liên tiếp” là mẫu *cục bộ*, đúng điểm mạnh của tích chập; XGBoost tốt nhất vì phần lớn dòng test không có lịch sử và ở đó mô hình cây dùng đặc trưng dạng bảng hiệu quả hơn.
- **Mô hình nhanh nhất?** MLP (Bảng 9). Trong các mô hình chuỗi: ở A, GRU/LSTM 1 tầng và CNN đơn giản ngang nhau ($\approx 0,4$ s/epoch); ở B, CNN nhanh hơn RNN $\approx 25\%$.
- **Mô hình ổn định nhất?** Độ lệch chuẩn qua 3 seed của các cấu hình đều nhỏ (0,001–0,008; riêng B_gru_cnt 0,015), nhỏ hơn nhiều so với khoảng cách A–B. CNN có BN + Dropout trên B cho đường validation mượt và khoảng cách train–val nhỏ nhất (mục 7).
- **Kết quả có hợp lý không?** Có. Accuracy luôn $> 98\%$ nhưng vô nghĩa (đoán toàn *white* đã đạt 98,6%), nên macro F1 được dùng làm độ đo chính. Lớp *white* luôn có F1 $\approx 0,99$; các họ có nhiều địa chỉ lặp lại được cải thiện mạnh nhờ lịch sử, các họ dùng địa chỉ một lần thì không — đúng với phân

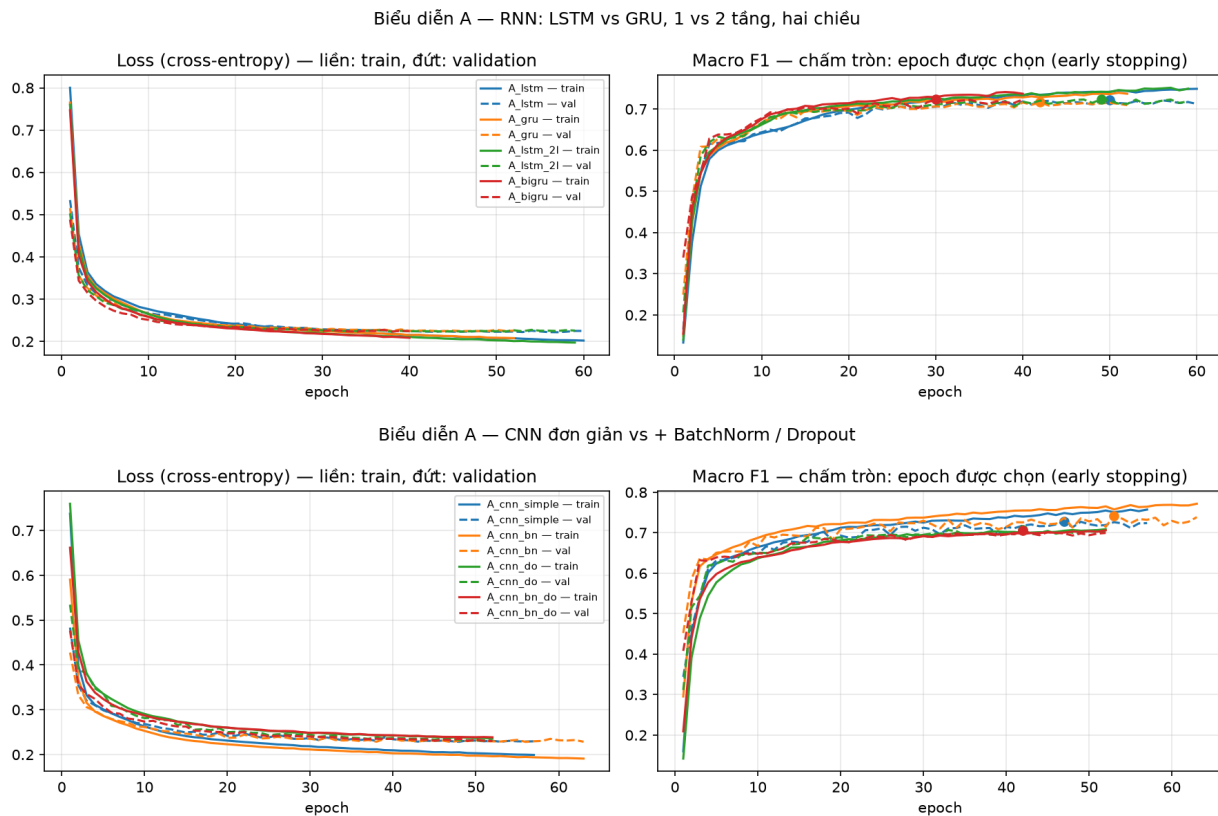
tích dữ liệu ở mục 2. Cách chia theo địa chỉ đảm bảo kết quả của B không đến từ việc “nhớ” địa chỉ đã thấy khi huấn luyện.

7 Phân tích overfitting / underfitting

Cách đọc. Các đường cong dưới đây vẽ loss (cross-entropy) và macro F1 theo epoch trên tập train (đã undersample) và `val_us` (validation undersample *cùng tỉ lệ*), nên hai đường so sánh trực tiếp được. Chấm tròn là epoch được early stopping giữ lại. Lưu ý: số liệu train là trung bình *trong lúc huấn luyện* (dropout đang bật, trọng số thay đổi trong epoch), nên với mô hình có dropout, F1 train có thể thấp hơn F1 validation. Bảng 10 tóm tắt các dấu hiệu chính.

Bảng 10: Chẩn đoán overfit/underfit (seed 42; macro F1 và loss trên tập train đã undersample và `val_us`). Ep. min loss: epoch có val loss nhỏ nhất; Ep. tốt nhất: epoch có val macro F1 lớn nhất (được giữ lại).

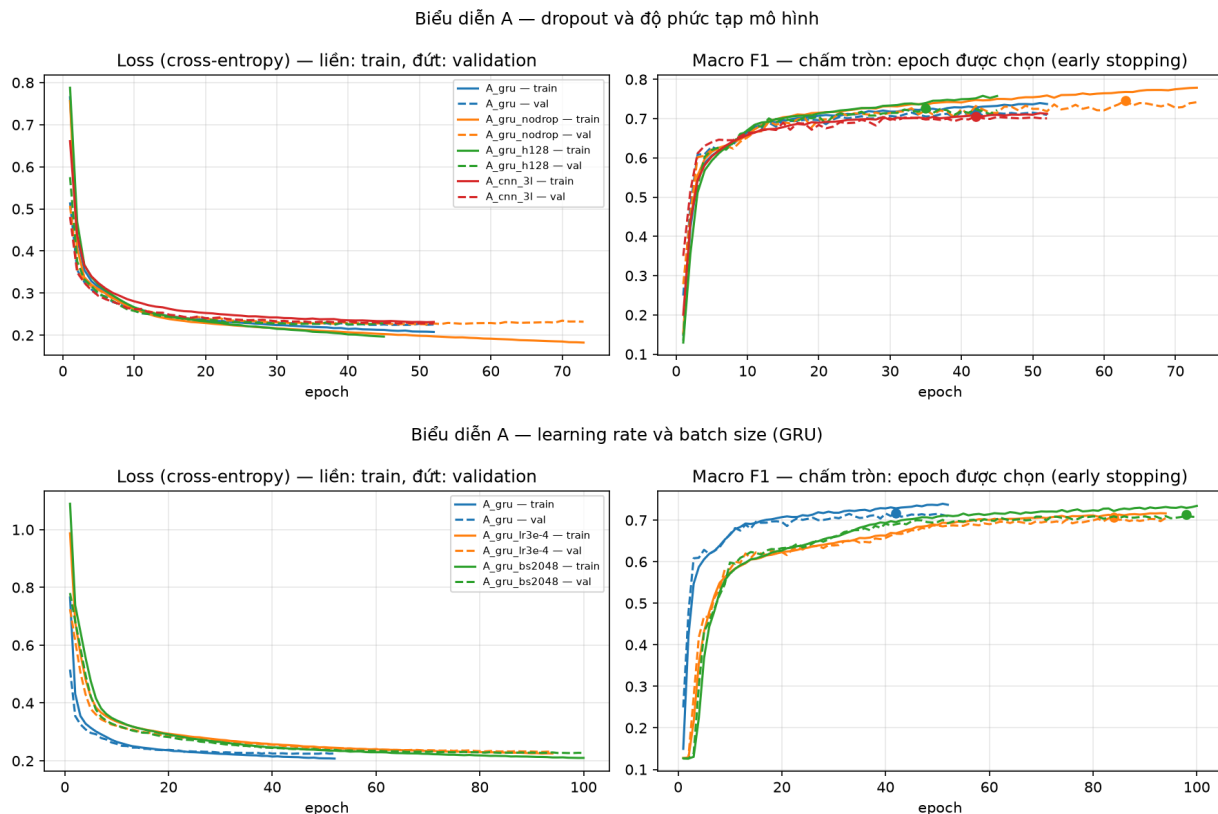
Cấu hình	Epoch			Tại epoch tốt nhất			Tại epoch cuối		
	min loss	tốt nhất	cuối	F1 train	F1 val	chênh	loss train	loss val	F1 chênh
A_lstm	57	50	60	0.742	0.722	+0.020	0.202	0.225	+0.037
A_gru	46	42	52	0.729	0.717	+0.012	0.208	0.225	+0.028
A_gru_nodrop	51	63	73	0.768	0.746	+0.022	0.183	0.232	+0.037
A_gru_h128	38	35	45	0.740	0.726	+0.014	0.197	0.229	+0.036
A_gru_lr3e-4	92	84	94	0.711	0.706	+0.004	0.226	0.231	+0.011
A_gru_bs2048	98	98	100	0.730	0.713	+0.017	0.210	0.227	+0.027
A_cnn_simple	48	47	57	0.749	0.728	+0.021	0.199	0.230	+0.034
A_cnn_bn	47	53	63	0.758	0.741	+0.016	0.191	0.229	+0.033
A_cnn_do	50	42	52	0.699	0.708	-0.009	0.236	0.232	+0.006
A_cnn_bn_do	52	42	52	0.697	0.705	-0.008	0.239	0.230	+0.007
A_mlp	49	42	52	0.713	0.722	-0.009	0.220	0.226	+0.015
B_gru	22	53	63	0.911	0.855	+0.056	0.095	0.169	+0.063
B_gru_2l	61	53	63	0.915	0.881	+0.034	0.094	0.152	+0.041
B_gru_L32	29	26	36	0.895	0.845	+0.050	0.105	0.162	+0.063
B_cnn_simple	21	12	22	0.867	0.831	+0.036	0.117	0.159	+0.071
B_cnn_bn_do	31	22	32	0.863	0.847	+0.016	0.131	0.136	+0.035
B_cnn_k5	65	66	76	0.890	0.861	+0.029	0.118	0.132	+0.036
B_mlp	96	88	98	0.872	0.845	+0.026	0.122	0.142	+0.035



Hình 7: Biểu diễn A: RNN (trên) và CNN đơn giản / + BatchNorm / + Dropout (dưới).

7.1 Biểu diễn A: giới hạn bởi thông tin, không phải bởi năng lực mô hình

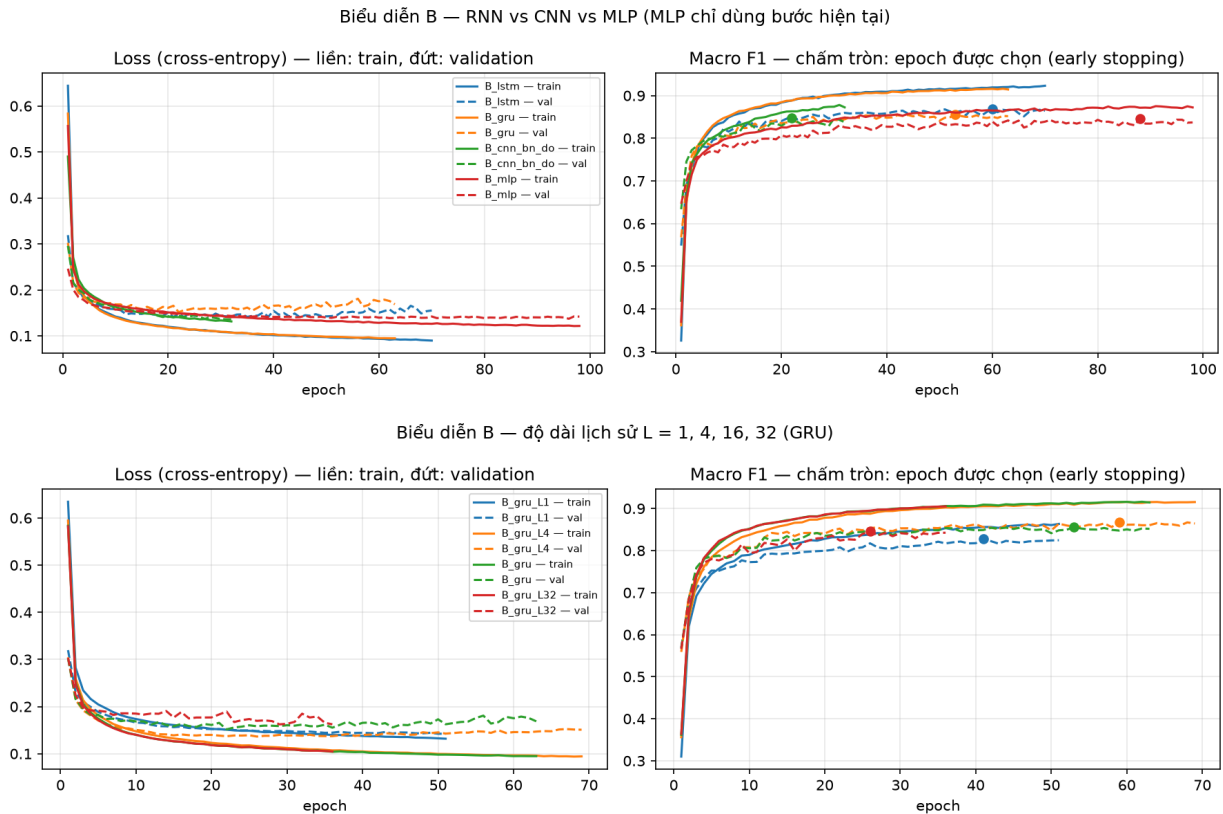
- Khoảng cách train-val rất nhỏ ($\leq 0,02$ macro F1; loss 0,20 so với 0,23) ở mọi cấu hình, và **tăng năng lực mô hình** (2 tầng, 128 units, hai chiều, 3 lớp conv, gấp đôi filters) **không cải thiện** validation. Đây không phải overfitting, cũng không phải underfitting do mô hình quá nhỏ: một dòng đơn lẻ không chứa đủ thông tin để tách ransomware khỏi white (sai số không giảm được — *irreducible error*). XGBoost trên cùng thông tin cũng chỉ đạt 0,575.
- Overfit nhẹ:** `A_gru_nodrop` và `A_cnn_bn` có F1 train cao nhất (0,77) và val loss đạt cực tiểu sớm hơn epoch tốt nhất (51 so với 63; 47 so với 53) rồi tăng lại (0,232 và 0,229 ở epoch cuối) trong khi train loss tiếp tục giảm — dấu hiệu kinh điển của overfitting. Early stopping dừng kịp; trên test, mức overfit này gần như không gây hại.
- Underfit:**
 - `A_cnn_do`, `A_cnn_bn_do`, `A_mlp`: F1 train *thấp hơn* validation, cả hai đều thấp hơn bản không dropout. Dropout 0,3 với mô hình chỉ ≈ 23 nghìn tham số là điều chuẩn thừa.
 - `A_gru_lr3e-4`: learning rate nhỏ $\Rightarrow$ đường cong tăng chậm, epoch tốt nhất là 84, cả train và val đều thấp hơn (0,71/0,71). `A_gru_bs2048`: batch lớn $\Rightarrow$ số bước cập nhật/epoch giảm 4 lần, epoch tốt nhất 98/100 — *chưa hội tụ* khi hết ngân sách epoch (Hình 8).



Hình 8: Biểu diễn A: ảnh hưởng của dropout và độ phức tạp (trên); learning rate và batch size (dưới).

7.2 Biểu diễn B: overfitting rõ hơn và vai trò của điều chuẩn

- Với lịch sử, mô hình có nhiều thông tin hơn nên F1 train lên tới 0,91–0,92, và **khoảng cách train–val lớn hơn hẳn** (GRU: 0,911 so với 0,855; val loss 0,169 so với train 0,095 ở epoch cuối). Val loss của B_gru đạt cực tiểu ở epoch 22 rồi tăng dần, trong khi val F1 vẫn đi ngang/tăng nhẹ đến epoch 53: mô hình tiếp tục *tự tin quá mức* vào các dự đoán sai (loss tăng) dù số dự đoán đúng không giảm (F1 không giảm). Vì đề bài đánh giá bằng F1, early stopping theo F1 thay vì loss là lựa chọn hợp lý; trọng số lớp w hiệu chỉnh sau cũng bù một phần cho xác suất bị lệch.
- Nguyên nhân overfit dễ hơn: các dòng của cùng một địa chỉ có lịch sử chồng lấp nhau, nên số mẫu “độc lập” thực sự ít hơn nhiều so với 126 nghìn dòng (trong toàn bộ dữ liệu, 3.403 địa chỉ CryptoWall/CryptoLocker tạo ra 21.705 dòng, trong khi 9.223 dòng Cerber đến từ 9.177 địa chỉ); mô hình có thể học thuộc mẫu riêng của từng địa chỉ trong tập train.
- CNN không điều chuẩn** (B_cnn_simple) overfit nhanh nhất: epoch tốt nhất là 12, chênh F1 tăng từ 0,036 lên 0,071 ở epoch cuối, test chỉ 0,686. **Thêm BatchNorm + Dropout** (B_cnn_bn_do): train/val loss gần trùng nhau (0,131/0,136), chênh F1 0,016 — tổng quát hoá tốt nhất — và test tăng lên 0,724. Đây là điểm khác so với biểu diễn A: *điều chuẩn chỉ có ích khi mô hình thực sự có xu hướng overfit*.
- GRU 2 tầng (dropout giữa các tầng) có khoảng cách nhỏ hơn GRU 1 tầng (0,034 so với 0,056) và tốt hơn trên test — dropout giữa tầng đóng vai trò điều chuẩn. L=32 overfit sớm hơn (dừng ở epoch 26).
- B_mlp (chỉ bước hiện tại) gần như không overfit nhưng cả train và val đều thấp (0,87/0,85): *underfit* do thiếu thông tin lịch sử, không phải do mô hình nhỏ.



Hình 9: Biểu diễn B: RNN vs CNN vs MLP chỉ dùng bước hiện tại (trên); GRU với độ dài lịch sử $L = 1, 4, 16, 32$ (dưới).

7.3 Biện pháp đã áp dụng và đề xuất

- **Đã áp dụng:** early stopping (patience 10, khôi phục trọng số tốt nhất) cho mọi mô hình; Dropout ở đầu phân loại và giữa các tầng RNN; BatchNorm và Dropout cho CNN; cắt gradient; undersampling + hiệu chỉnh trọng số lớp trên validation.
- **Với overfit (biểu diễn B):** weight decay / L2 (chưa thử — `weight_decay=0`); giảm số lần xuất hiện được dùng của các địa chỉ có rất nhiều dòng (để mỗi địa chỉ đóng góp cân bằng hơn); làm nhiễu lịch sử (bỏ ngẫu nhiên một số bước — một dạng tăng cường dữ liệu cho chuỗi); label smoothing để giảm hiện tượng tự tin quá mức.
- **Với underfit:** tăng learning rate hoặc dùng lịch giảm learning rate (cosine/one-cycle) thay vì lr nhỏ cố định; khi dùng batch lớn thì tăng lr tương ứng hoặc tăng ngân sách epoch; giảm dropout cho mô hình nhỏ. Với biểu diễn A, cách duy nhất để vượt giới hạn là *thêm thông tin* (như biểu diễn B), không phải mô hình lớn hơn.

8 Hạn chế và hướng phát triển

8.1 Hạn chế

- **Dữ liệu là dạng bảng, không phải chuỗi tự nhiên.** Biểu diễn A tạo “chuỗi” bằng cách xếp 11 đặc trưng theo một thứ tự tùy ý; RNN/CNN không có lợi thế nào so với MLP và thua mô hình cây. Biểu diễn B là chuỗi thật (theo thời gian) nhưng chỉ có ích cho địa chỉ được dùng lại — 90% dòng test là địa chỉ xuất hiện lần đầu, chuỗi chỉ dài 1.
- **Lớp white được lấy mẫu** (khoảng 1.000 dòng/ngày trong dữ liệu gốc), nên “lịch sử” của một địa chỉ white bị thưa hơn thực tế, và đặc trưng “số lần đã xuất hiện” có thể phân biệt hai lớp tốt hơn so với khi triển khai thật trên toàn bộ chuỗi khối. Kết quả của biểu diễn B có thể lạc quan so với thực tế.

- **Nhãn ransomware không đầy đủ:** chỉ các địa chỉ đã được các nghiên cứu trước xác định; nhiều dòng *white* có thể là ransomware chưa được phát hiện, làm giảm precision quan sát được.
- **Hai cách chia khác nhau:** biểu diễn A dùng chia ngẫu nhiên (để so sánh với Bài 1) — 54,7% dòng ransomware trong test có địa chỉ đã xuất hiện trong train, nên kết quả A có rõ ràng qua địa chỉ. Bảng 6 đã kiểm chứng: trên cách chia theo địa chỉ, A cho kết quả gần như giống hệt, nên kết luận không thay đổi.
- **Ước lượng phương sai còn hạn chế:** chỉ 3 seed cho các cấu hình chính (1 seed cho các biến thể còn lại), và chỉ một cách chia dữ liệu (không cross-validation). Khác biệt $<0,01$ giữa các cấu hình không nên coi là có ý nghĩa.
- **Early stopping theo val_us** (31,6 nghìn dòng, chỉ $\approx 6,6$ nghìn dòng ransomware) có nhiều; nhiều cấu hình A cùng seed dừng ở cùng epoch (42) vì dùng chung thứ tự batch, cho thấy chọn epoch phụ thuộc phần nào vào dao động ngẫu nhiên.
- **Siêu tham số chưa tối ưu toàn diện:** thay đổi từng yếu tố một so với cấu hình gốc (không tìm kiếm lưới/Bayes); không thử recurrent dropout, weight decay, lịch learning rate. Thời gian/epoch trong bảng chính bị ảnh hưởng do chạy song song (đã đo lại riêng ở Bảng 9).

8.2 Hướng phát triển

- **Kết hợp hai thế mạnh:** mô hình cây cho đặc trưng của dòng hiện tại và RNN/CNN cho lịch sử — ví dụ dùng embedding lịch sử do CNN-B học được làm đặc trưng đầu vào cho XGBoost, hoặc ensemble xác suất của hai mô hình (XGBoost tốt hơn ở địa chỉ mới, CNN-B tốt hơn ở địa chỉ có lịch sử dài — Bảng 7).
- **Mô hình đồ thị:** ransomware chuyển tiền qua nhiều địa chỉ; cần dữ liệu giao dịch gốc (cạnh giữa các địa chỉ) để dùng Graph Neural Network — giúp cả với địa chỉ dùng một lần như Cerber/Locky.
- Transformer trên lịch sử địa chỉ (self-attention với mã hoá khoảng cách thời gian); mô hình FT-Transformer [8] cho biểu diễn A.
- Tối ưu siêu tham số tự động (Optuna), nhiều seed hơn, cross-validation theo địa chỉ, và hiệu chỉnh xác suất (temperature scaling) trước khi tìm ngưỡng/trọng số lớp.
- Đánh giá theo thời gian (train trên các năm trước, test trên năm sau) để mô phỏng triển khai thực tế và kiểm tra khả năng phát hiện họ ransomware mới.

Tài liệu tham khảo

- [1] C. G. Akcora, Y. Li, Y. R. Gel, M. Kantarcioglu. *BitcoinHeist: Topological Data Analysis for Ransomware Detection on the Bitcoin Blockchain*. IJCAI-PRICAI 2020. Bộ dữ liệu: UCI Machine Learning Repository, *BitcoinHeistRansomwareAddressDataset*.
- [2] S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory*. Neural Computation 9(8), 1997.
- [3] K. Cho và cộng sự. *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP 2014 (GRU).
- [4] Y. LeCun, Y. Bengio. *Convolutional Networks for Images, Speech, and Time Series*. The Handbook of Brain Theory and Neural Networks, 1995.
- [5] S. Ioffe, C. Szegedy. *Batch Normalization*. ICML 2015.
- [6] N. Srivastava và cộng sự. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. JMLR 15, 2014.
- [7] D. P. Kingma, J. Ba. *Adam: A Method for Stochastic Optimization*. ICLR 2015.

- [8] Y. Gorishniy và cộng sự. *Revisiting Deep Learning Models for Tabular Data*. NeurIPS 2021 (ý tưởng mã hoá mỗi đặc trưng thành một token).
- [9] L. Grinsztajn, E. Oyallon, G. Varoquaux. *Why do tree-based models still outperform deep learning on tabular data?* NeurIPS 2022 Datasets and Benchmarks.
- [10] T. Chen, C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. KDD 2016.
- [11] Thư viện: PyTorch 2.11, scikit-learn 1.x, XGBoost, pandas, NumPy, Matplotlib.

A Mã nguồn và cách chạy lại

Cấu trúc.

- `final/data.py` — đọc dữ liệu, tạo nhãn 7 lớp và đặc trưng, chia ngẫu nhiên / theo địa chỉ, undersampling, dựng lịch sử địa chỉ (biểu diễn B), chuẩn hoá (chỉ tính trên train).
- `final/models.py` — FeatureTokenizer (biểu diễn A), RNNNet (LSTM/GRU, 1–2 tầng, một/hai chiều, last/mean/attention, pack_padded_sequence), CNNNet (Conv1D, BN, Dropout, Max/Avg pooling, Flatten/GAP/GMP có mặt nạ), MLPNet.
- `final/configs.py` — toàn bộ cấu hình thực nghiệm (tên, kiến trúc, siêu tham số).
- `final/train.py` — huấn luyện với early stopping, ghi lịch sử từng epoch, hiệu chỉnh trọng số lớp trên validation, đánh giá test; lưu `history.json`, `result.json`, `model.pt`, `pred_test.npy` vào `outputs/final/<biểu diễn>_<cách chia>_<cấu hình>/.`
- `final/baselines.py` — XGBoost (GPU) trên hai cách chia với 3 mức thông tin lịch sử.
- `final/benchmark.py` — đo tốc độ huấn luyện từng mô hình khi chạy riêng.
- `final/report.py` — tổng hợp kết quả thành các bảng LaTeX/CSV (`outputs/final/tables/`) và hình (`outputs/final/figures/`) dùng trong báo cáo.
- `final_project.ipynb` — notebook trình bày đầu–cuối: thống kê dữ liệu, minh hoạ hai biểu diễn, in kiến trúc, bảng kết quả, đường cong huấn luyện, ma trận nhầm lẫn, dự đoán mẫu và phân tích lỗi. Mặc định notebook *nap kết quả đã huấn luyện*; đặt `RETRAIN = True` để huấn luyện lại một số cấu hình tiêu biểu ngay trong notebook.

Cách chạy. Cần Python 3.12, các thư viện trong `requirements.txt` (thêm PyTorch có CUDA nếu dùng GPU) và tệp `BitcoinHeistData.csv` ở thư mục gốc. Siêu tham số XGBoost được đọc từ `exp_out/best_params.json` (kết quả tinh chỉnh của Bài 1).

```
pip install -r requirements.txt
python -m final.train --rep A           # mọi cấu hình A, chia ngẫu nhiên
python -m final.train --rep B           # mọi cấu hình B, chia theo địa chỉ
python -m final.train --rep A --split group --only A_gru+A_lstm+A_mlp+A_cnn_bn_do
python -m final.train --rep B --seeds 43+44 --only B_gru+B_cnn_bn_do # seed khác
python -m final.baselines               # XGBoost
python -m final.benchmark               # đo tốc độ (khi GPU rảnh)
python -m final.report                  # bảng + hình
cd report && latexmk -xelatex main.tex  # biên dịch báo cáo
```

Có thể chạy nhiều tiến trình `final.train` song song với cùng tham số: mỗi tiến trình “nhận” một cấu hình chưa chạy (qua một tệp khoá) nên không chạy trùng. Thời gian: $\approx 0,5\text{--}2,5$ phút/cấu hình A và $0,5\text{--}7$ phút/cấu hình B trên GPU RTX PRO 4000 (khi chạy song song).

Tái lập. Chia dữ liệu và undersampling luôn dùng `random_state=42`; seed huấn luyện (42, 43, 44) cố định khởi tạo trọng số và thứ tự batch. Do một số phép toán cuDNN không tắt định, chạy lại có thể khác ở chữ số thập phân thứ 3 — nhỏ hơn độ dao động giữa các seed.